

CAD-MLLM: Unifying Multimodality-Conditioned CAD Generation With MLLM

David Chen, Karthick Raja BG, Yizhuo Di, Chia Hui Yen (Audit).
Carnegie Mellon University

Abstract

This project introduces CAD-MLLM, a unified framework for generating parametric CAD models conditioned on text, images, and point clouds. Traditional Computer-Aided Design (CAD) workflows require manual, expert-intensive manipulation of parameters, creating a barrier between high-level design intent and geometric realization. To address this, we propose a Large Language Model (LLM) based architecture that treats CAD generation as a sequence prediction task, serializing topological data into parametric command sequences. We constructed a large-scale multimodal dataset by extending the DeepCAD framework, using a novel “Intelligent Truncation” algorithm to generate over 197,000 valid partial-to-complete design pairs with synchronized B-Rep, render, and point cloud modalities. The model is trained using a curriculum learning strategy, progressing from text-only alignment to randomized multimodal fusion, utilizing Low-Rank Adaptation (LoRA) for efficient fine-tuning. Experimental results demonstrate that text-only conditioning achieves a 90% success rate in generating valid STEP files, while multimodal inputs significantly enhance fine-grained entity sequence accuracy. This work advances the capability for intuitive, text-driven 3D editing and generation, bridging the gap between natural language and precise geometric construction.

1 Introduction

Computer-Aided Design (CAD) is the standard for modern engineering and manufacturing, yet the process of creating and editing parametric models remains complex and non-intuitive. Traditional workflows rely on Boundary Representation (B-rep) structures—graphs of faces, edges, and vertices—which are difficult to generate directly using standard autoregressive AI models due to their non-sequential nature. While generative AI has made strides in unstructured 3D representations like meshes, enabling precise, text-driven editing of parametric CAD models remains a significant challenge.

The primary goal of this project is to democratize 3D design by enabling a model to accept natural language instructions, 2D images, or 3D point clouds and generate topologically valid parametric CAD sequences. By treating CAD models as sequences of construction operations (e.g., “Sketch,” “Extrude”), we leverage the reasoning capabilities of pre-trained Large Language Models (LLMs) to interpret high-level semantic intent.

A critical bottleneck in this domain is the lack of aligned multimodal datasets. Existing large-scale repositories like DeepCAD provide parametric JSON sequences but lack the corresponding visual and geometric data necessary for training robust multimodal agents. To overcome this, we developed a comprehensive data generation pipeline that synthesizes B-Reps, multi-view renders, and point clouds from raw command sequences. Furthermore, we introduced an “Intelligent Truncation” algorithm to generate geometrically valid partial designs, enabling the model to learn autocompletion logic.

In this report, we present the methodology and evaluation of CAD-MLLM. We detail our command sequence representation, multimodal feature alignment strategy, and curriculum training approach. Our evaluation focuses on both geometric validity (STEP conversion rates, dangling edges) and sequence accuracy, providing insights into the trade-offs between modality richness and generation stability.

2 Related Work

The domain of generative CAD has shifted from synthesizing discrete 3D representations, such as voxels and meshes, to generating parametric command sequences that preserve design history. **DeepCAD** [3] pioneered this approach by treating CAD operations as a language sequence, employing a Transformer-based autoencoder to learn a latent representation of construction commands [3]. This allowed for the generation of editable CAD models rather than static shapes.

Subsequent works focused on conditioning these generative processes on semantic inputs. **Text2CAD** [2] introduced a framework to generate parametric CAD sequences directly from natural language descriptions. By creating a dataset of over 660,000 text annotations using Large Language Models (LLMs), Text2CAD enabled the synthesis of models from both abstract and detailed prompts, effectively mapping textual intent to geometric construction [2].

Challenging the reliance on task-specific command sequences, **Text-to-CadQuery** [4] proposed a new paradigm by generating executable Python code directly from natural language. By augmenting the Text2CAD dataset with 170,000 CadQuery scripts, this work demonstrated that fine-tuning general-purpose LLMs (such as Qwen and Mistral) allows models to leverage their pre-trained coding capabilities. This approach not only simplified the generation pipeline by removing intermediate representations but also established that model performance scales effectively with parameter size, significantly outperforming custom transformer architectures in geometric accuracy [4].

To address visual conditioning, **CAD-Coder** [1] proposed a Vision-Language Model (VLM) fine-tuned to generate executable CadQuery code from input images. Unlike methods that output abstract tokens, CAD-Coder leverages the code-generation capabilities of foundation models (such as LLaVA) to ensure 100% syntactic validity and high geometric accuracy, demonstrating the efficacy of VLMs in engineering design [1].

Our work, **CAD-MLLM**, unifies these modalities into a comprehensive framework. While previous methods often specialized in either text or image inputs, CAD-MLLM integrates text, single-view images, and point clouds within a single Multimodal Large Language Model [5]. By employing a curriculum learning strategy and an “Intelligent Truncation” algorithm for partial sequence training, CAD-MLLM enables robust design autocompletion and editing across multiple input types [5].

3 Dataset Generation

To enable multi-modal learning and support our novel autocompletion objective, we constructed a comprehensive synthetic dataset extending the DeepCAD framework. While the original DeepCAD work provides large-scale parametric CAD sequences, the publicly available dataset lacks the synchronized multi-modal alignment (specifically pre-computed B-Reps and Renders) required for training multi-modal agents. To address this, we developed a generation pipeline that synthesizes these missing modalities and creates valid partial design sequences.

3.1 Multimodal Augmentation Pipeline

We constructed our dataset starting from a representative **10% subset (58,653 models)** of the `cad_json` data provided by the official DeepCAD repository [3]. This raw data consists solely of JSON-based parametric command sequences (e.g., Sketch and Extrude parameters).

To unify semantic, visual, and geometric representations, we adapted the data processing scripts provided by Wu et al. [3], utilizing the **pythonocc** (OpenCascade) geometry kernel to synthetically generate the missing ground truth modalities. For every training instance, our pipeline generates:

- **B-Rep Geometry (STEP):** We compiled the raw JSON commands into precise Boundary Representation (B-Rep) files.
- **Point Clouds:** Using the standard sampling logic from the DeepCAD repository, we generated high-resolution surface point clouds from the B-Rep geometry to support 3D geometric encoders.
- **Multi-View Renderings:** To provide strong visual conditioning, we implemented a custom rendering script to capture four standardized viewpoints (Front, Top, Side, Isometric) for each model.

This engineering effort resulted in a unified dataset where every design is represented across textual (command sequence), visual (image), and geometric (B-Rep/PCD) domains.

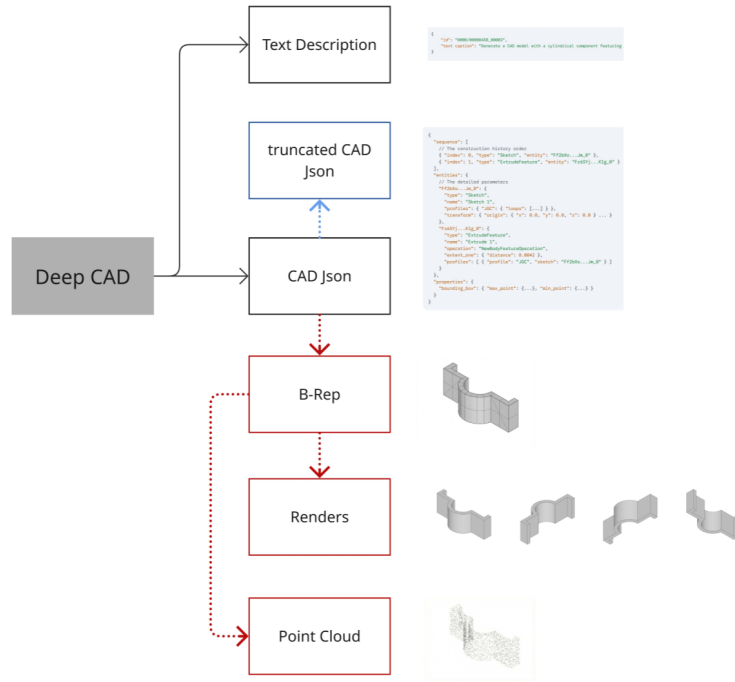


Figure 1: Multimodal Data Generation Pipeline. We process the raw JSON sequences from a 10% DeepCAD subset [3] using OpenCascade to synthetically generate B-Reps, Multi-view Renders, and Point Clouds for every training instance.

3.2 Intelligent Truncation for Autocompletion

A core contribution of this work is the **Intelligent Truncation Algorithm**, designed to generate high-quality training data for design autocompletion. Training a model to complete designs requires pairs of *partial* inputs and *complete* targets. However, simply truncating a parametric sequence at a random index often results in invalid geometry (e.g., a sketch without a plane, or a fillet without an edge).

Our algorithm addresses this by enforcing **geometric validity** at every step:

1. **Operation Boundary Detection:** The algorithm first identifies valid stopping points in the command sequence (e.g., immediately after a `ExtrudeFeature` is completed).
2. **Dependency Tracing:** For a chosen cut point, the algorithm recursively traces the dependency graph of the active entities. It identifies all sketches, reference planes, and profiles required to support the geometric features present in the partial sequence.
3. **Entity Cleanup:** Any “orphan” entities (commands that are defined but never used in the partial sequence) are pruned to ensure a clean, efficient JSON structure.

This process allows us to derive multiple valid partial variations (e.g., 25%, 50%, 75% complete) from a single source model, effectively teaching the model the logic of incremental construction.

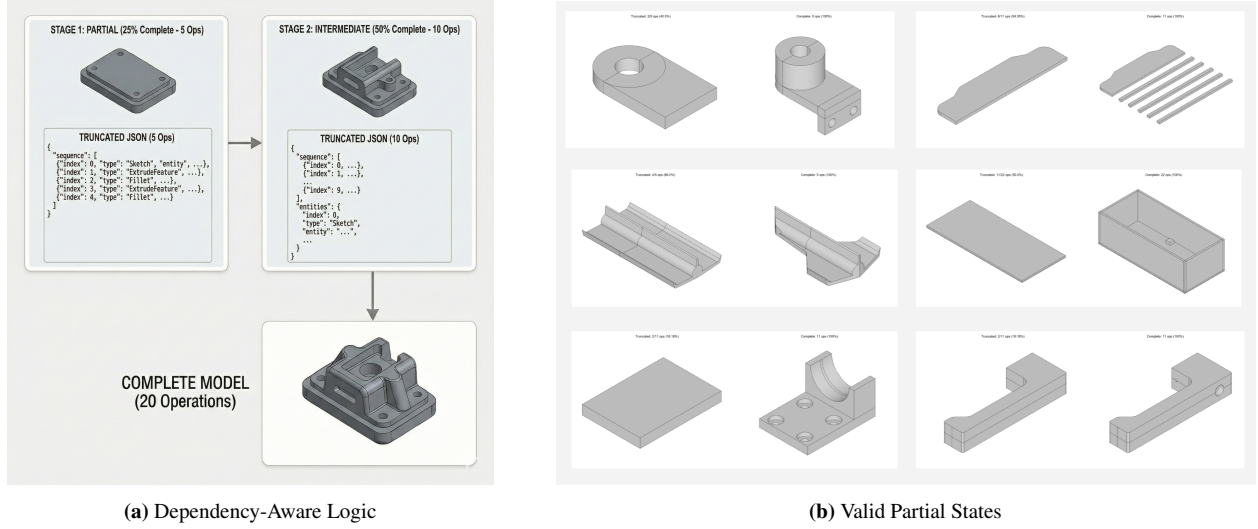


Figure 2: Intelligent Truncation Process. (a) Our algorithm recursively traces dependencies to ensure that partial JSON sequences remain geometrically valid. (b) Visual comparison of generated partial states (left/center) versus the final complete model (right), demonstrating geometric consistency at every stage.

3.3 Dataset Statistics and Amplification

By applying Intelligent Truncation, we significantly amplified the training signal available from our 10% subset. The algorithm generated an average of **2.37 truncated versions per model**, effectively amplifying our dataset size by a factor of **3.37 $\times$** . The final dataset consists of **197,546 valid training examples**, expanded from the original 58,653 source models.

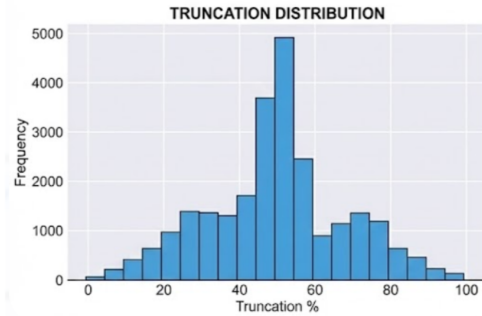


Figure 3: Truncation Distribution. The dataset covers the full range of design completeness (1%-99%), ensuring balanced training for all stages.

Table 1: Dataset Comparison.

Metric	DeepCAD	Ours	Gain
Distinct Models	58,653	58,653	1.0 $\times$
Sequences	58,653	197,546	3.37$\times$
B-Rep (STEP)	–	58,653	–
Point Clouds	–	58,653	–
Renderings	–	790,184	–
Avg. Ver/Model	1.0	2.37	2.37 $\times$
Completeness	100%	1–99%	–

4 Methodology

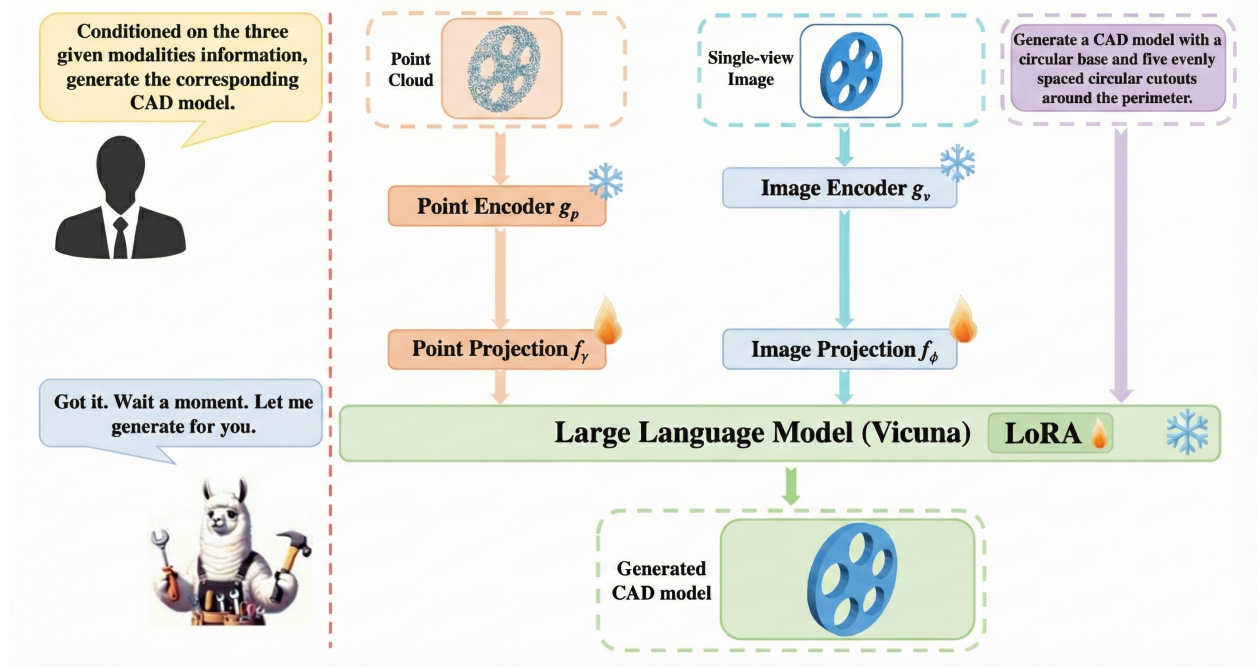


Figure 4: The architecture of the mode.

In this section, we present CAD-MLLM (), a comprehensive framework designed to generate parametric CAD models conditioned on diverse modalities, including text, images, and point clouds. We first define the command sequence representation used to serialize CAD topology. Subsequently, we detail the multimodal feature alignment mechanism that projects various input signals into the Large Language Model (LLM) embedding space. Finally, we describe the training objectives and strategies, including Low-Rank Adaptation (LoRA) and curriculum learning.

4.1 Command Sequence-Based CAD Representation

Traditional CAD models are typically stored in Boundary Representation (B-rep) formats, which describe geometry through topological connections (faces, edges, vertices) rather than construction history. B-rep data is inherently non-sequential and graph-based, making it unsuitable for direct autoregressive generation by standard LLMs.

To address this, we adopt a *parametric command sequence* representation. We define a CAD model M as a sequence of construction operations S . Each operation corresponds to a modeling command (e.g., `Sketch`, `Extrude`) defined by specific parameters. The sequence is formally denoted as:

$$S = \{c_1, c_2, \dots, c_N\} \quad (1)$$

where N is the total number of commands, and each c_i represents a tuple containing the command type t_i (e.g., `Line`, `Arc`, `Circle`) and its associated geometric parameters p_i (e.g., coordinates, radius, extrusion depth).

4.2 Multimodal Feature Alignment

The core of CAD-MLLM is its ability to condition generation on multimodal inputs. We utilize a pre-trained LLM as the backbone and align separate modality encoders into its input embedding space.

4.2.1 Text Conditioning

Natural language inputs describe the desired semantic attributes of the CAD model. These text descriptions T are processed directly by the LLM’s native tokenizer and embedding layer, producing a sequence of text embeddings

$E_T \in \mathbb{R}^{L_T \times D}$, where L_T is the sequence length and D is the embedding dimension.

4.2.2 Visual Conditioning

For image-conditioned generation, we employ a pre-trained visual encoder to extract feature representations from multi-view renderings of the CAD object. Given an input image I , the visual encoder produces a feature map F_I . To bridge the dimensionality gap between the visual encoder and the LLM, we apply a learnable projection layer P_v :

$$E_I = P_v(F_I) \quad (2)$$

where $E_I \in \mathbb{R}^{L_I \times D}$ represents the visual embeddings aligned with the LLM’s dimension D .

4.2.3 Point Cloud Conditioning

To incorporate 3D spatial information, we utilize a point cloud encoder. The input point cloud P is encoded into a global feature vector F_P . Similar to the visual pipeline, a projection layer P_p maps these features to the LLM space:

$$E_P = P_p(F_P) \quad (3)$$

resulting in point cloud embeddings $E_P \in \mathbb{R}^{L_P \times D}$.

4.3 Architecture and Training

The aligned embeddings from the conditional modalities are concatenated with the task prompt tokens to form the input sequence X_{in} . The model is trained to generate the target CAD command sequence S autoregressively.

4.3.1 Training Objective

We employ the standard Causal Language Modeling objective. The model maximizes the log-likelihood of the next token s_t in the command sequence given the conditional inputs $\mathcal{C}$ (where $\mathcal{C} \subset \{T, I, P\}$) and the preceding tokens $s_{<t}$:

$$\mathcal{L}(\theta) = - \sum_{t=1}^N \log P_{\theta}(s_t \mid s_{<t}, \mathcal{C}) \quad (4)$$

where θ represents the trainable parameters of the model.

4.3.2 Efficient Fine-Tuning and Curriculum Learning

To adapt the LLM to the CAD domain while preserving its reasoning capabilities, we utilize Low-Rank Adaptation (LoRA). We freeze the pre-trained weights of the LLM backbone and the modality encoders, optimizing only the projection layers and the LoRA rank decomposition matrices.

Furthermore, we implement a *curriculum learning* strategy. The model is initially trained on single-modality tasks to establish foundational alignment. In subsequent stages, we introduce mixed-modality samples, requiring the model to synthesize information from multiple sources to generate accurate CAD geometries.

5 Implementation Details

5.1 Training Setup

We implemented CAD-MLLM using the PyTorch framework and the HuggingFace Transformers library. All experiments were conducted on a Google Colab NVIDIA A100 (80GB) GPU.

To ensure parameter efficiency, we froze the majority of the LLM parameters and the pre-trained modality encoders (DINO V2). We only fine-tuned the projection layers and the Low-Rank Adaptation (LoRA) modules injected into the attention mechanisms of the LLM. The LoRA configuration was set with a rank $r = 16$, scaling factor $\alpha = 32$, and dropout rate of 0.05.

5.2 Optimization and Hyperparameters

We trained the model using the AdamW optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.95$. We employed a cosine annealing learning rate scheduler with a warm-up period comprising 3% of the total training steps. The peak learning rate was set to 2×10^{-4} . The global batch size was maintained at 16, utilizing gradient accumulation to handle memory constraints. The maximum sequence length is 4096.

5.3 Curriculum Training Strategy

Following our curriculum learning approach, the training was divided into two distinct stages:

1. **Stage I: Text-Only Alignment.** In the initial phase, the model is trained exclusively on Text-to-CAD pairs. Since the CAD command sequence can be viewed as a specialized language, this stage focuses on establishing a strong semantic connection between natural language descriptions and CAD construction operations. This stabilizes the LLM’s understanding of the valid syntax.
2. **Stage II: Randomized Multimodal Fusion.** In the second phase, we introduce the visual and point cloud modalities. To ensure robustness and flexibility, we employ a *random modality combination* strategy. For each training sample, the conditioning input $\mathcal{C}$ is constructed by randomly sampling a subset of the available modalities:

$$\mathcal{C} \subseteq \{T, I, P\}, \quad \mathcal{C} \neq \emptyset \quad (5)$$

where T , I , and P denote text, image, and point cloud features, respectively. This forces the model to learn to generate accurate geometries regardless of whether it is provided with a single modality (Image only) or a dense combination of all three (Text + Image + Point Cloud).

6 Evaluation

6.1 Evaluation Settings and Configurations

We conducted four evaluation experiments with varying checkpoints, token limits, and input modalities:

Eval ID	Checkpoint	Max Tokens	Input Modality	Temperature
Eval 1	stage2-epoch0-step140-20251128_-10,240 072200		Text Only	0.5
Eval 2	stage2-epoch0-step140-20251128_-2,048 072200		Text Only	0.5
Eval 3	stage3-epoch0-step100-20251128_-4,096 220651		Image + Point Cloud + Text	0.5
Eval 4	stage-3-4096-20251129_213538	4,096	Image + Point Cloud + Text	0.5

Table 2: Evaluation configurations across checkpoints and modalities.

Pipeline. All evaluation samples are filtered such that the ground-truth JSON sequence length satisfies $L_{GT} < 2048$ tokens. This constraint ensures that the model is evaluated on samples with reasonable complexity, given the training data characteristics. The evaluation pipeline then proceeds through seven sequential steps: (1) **Text Prompt to JSON**, where the model performs inference to generate CAD JSON sequences; (2) **JSON to STEP Export**, which converts these JSON sequences to STEP file format; (3) **STEP File Visualization**, in which 3D visualizations are rendered for qualitative assessment; (4) **Topology Evaluation**, where geometric and topological properties are validated; [3, 5] (5) **JSON Structure Validation**, which verifies the syntactic correctness of the generated JSON; (6) **Sequence Metrics Evaluation**, where sequence-level accuracy metrics are computed; and (7) **CAD Sequence Evaluation**, which performs deeper analysis of entity sequences when available. [2].

6.2 Evaluation Metrics

6.2.1 Topology Evaluation Metrics

STEP/Raw Conversion Rate. The conversion rate measures the percentage of successfully generated valid STEP files from raw model outputs:

$$\text{Conversion Rate} = \frac{N_{\text{valid STEP}}}{N_{\text{total}}} \times 100\%, \quad (6)$$

where $N_{\text{valid STEP}}$ is the number of outputs that successfully convert to STEP format, and N_{total} is the total number of evaluation samples.

Dangling Edge Length (DangEL). DangEL quantifies the total length of boundary edges that are not properly connected in the generated geometry:

$$\text{DangEL} = \sum_{e \in E_{\text{boundary}}} \|e\|, \quad (7)$$

where E_{boundary} is the set of dangling (non-manifold) edges. Lower values indicate better geometric consistency. The expected range is $[0, +\infty)$ (ideal: 0).

Self-Intersection Ratio (SIR). SIR measures the percentage of self-intersecting faces in the generated mesh:

$$\text{SIR} = \frac{|F_{\text{intersect}}|}{|F_{\text{total}}|} \times 100\%, \quad (8)$$

where $F_{\text{intersect}}$ is the set of self-intersecting faces and F_{total} is the total number of faces. The expected range is $[0\%, 100\%]$ (ideal: 0%).

Flux Enclosure Error (FluxEE). FluxEE evaluates the enclosure quality by measuring flux consistency across the mesh surface:

$$\text{FluxEE} = \left| \oint_S \mathbf{n} \cdot d\mathbf{A} \right|, \quad (9)$$

where $\mathbf{n}$ is the surface normal and S represents the mesh surface. For a properly enclosed volume, this integral should approach zero. The expected range is $[0, +\infty)$ (ideal: 0).

6.2.2 CAD Sequence Metrics

Entity Count Accuracy. Entity Count Accuracy measures how well the predicted number of CAD entities matches the ground truth:

$$\text{Entity Count Acc} = 1 - \frac{|N_{\text{pred}} - N_{\text{GT}}|}{\max(N_{\text{pred}}, N_{\text{GT}})}, \quad (10)$$

where N_{pred} is the number of predicted entities and N_{GT} is the ground-truth count. The range is $[0.0, 1.0]$ (1.0 = perfect match).

Type Sequence Accuracy. Type Seq Acc evaluates the accuracy of the entity type sequence using edit distance:

$$\text{Type Seq Acc} = 1 - \frac{D_{\text{edit}}(T_{\text{pred}}, T_{\text{GT}})}{\max(|T_{\text{pred}}|, |T_{\text{GT}}|)}, \quad (11)$$

where D_{edit} is the Levenshtein edit distance between predicted sequence T_{pred} and ground-truth sequence T_{GT} . The range is $[0.0, 1.0]$.

Type Distribution Similarity. Type Dist Sim computes the Jaccard similarity between the distribution of entity types:

$$\text{Type Dist Sim} = \frac{|D_{\text{pred}} \cap D_{\text{GT}}|}{|D_{\text{pred}} \cup D_{\text{GT}}|}, \quad (12)$$

where D_{pred} and D_{GT} are the multisets of entity types in predictions and ground truth, respectively. The range is $[0.0, 1.0]$.

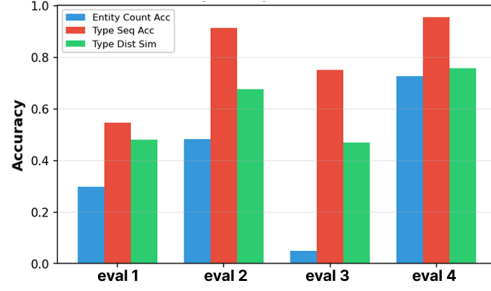


Figure 5: Summary of topology metrics for each evaluation configuration.

6.3 Results

Eval ID	Conv. Rate	Valid STEP	Total	DangEL ($\downarrow$)	SIR (% , $\downarrow$)	FluxEE ($\downarrow$)	Remarks
Eval 1	40.0 %	8	20	0	1	1.01e-17	Text-only, 10k tokens
Eval 2	90.0 %	45	50	0	1	4.11e-17	Text-only, 2k tokens
Eval 3	33.3 %	5	15	0	1	9.08e-18	Multi-modal, step100
Eval 4	60.0 %	9	15	0	1	2.90e-17	Multi-modal, 4096 tokens

Table 3: STEP conversion rates and topology metrics across evaluation configurations.

Eval ID	Entity Count Acc ($\uparrow$)	Type Seq Acc ($\uparrow$)	Type Dist Sim ($\uparrow$)	Remarks
Eval 1	0.2985	0.5469	0.4803	Text-only, 10k tokens
Eval 2	0.4823	0.9146	0.6757	Text-only, 2k tokens
Eval 3	0.05	0.75	0.4686	Multi-modal, step100
Eval 4	0.7273	0.9545	0.7576	Multi-modal, 4096 tokens

Table 4: Sequence-level accuracy metrics.

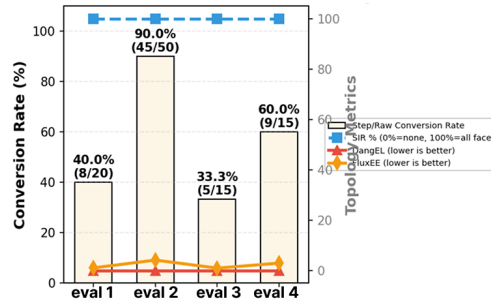


Figure 6: Summary of sequence accuracy metrics for each evaluation configuration.

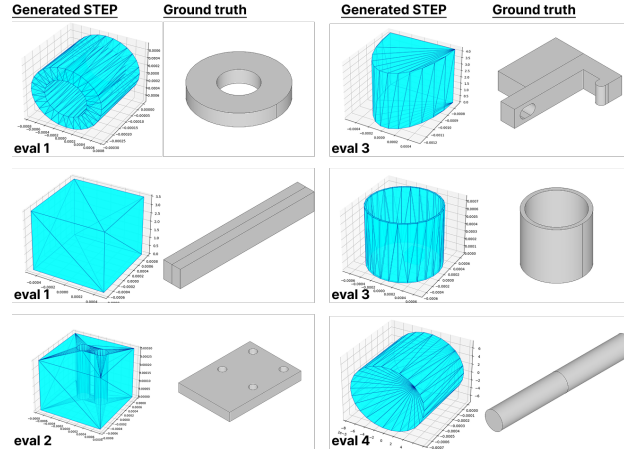


Figure 7: Visualization Examples.

6.4 Analysis and Discussion

Overall, the evaluation results show that our current CAD-MLLM pipeline can reliably generate simple CAD geometries, but still struggles with robustness and stability on more complex samples. In terms of modalities, text-only configurations tend to achieve higher STEP conversion rates (on average around 90%) but lower sequence-level accuracy, suggesting that they produce simpler, more syntactically valid outputs at the cost of geometric detail. In contrast, multi-modal runs that include images and point clouds achieve lower conversion rates (around 60%) but higher entity and type sequence accuracy, indicating that richer inputs help the model capture more detailed structure, but also make the generation process more fragile.

The maximum token length also appears to affect performance. When we allow a larger token budget (e.g., 10,240 tokens), the model can express more detailed sequences but is more likely to produce overly long or partially inconsistent outputs, which hurts conversion rate and increases topology errors. With a shorter token limit (e.g., 2,048 tokens), the model behaves more conservatively, leading to slightly higher stability but sometimes missing fine-grained constraints or entities. This reflects a trade-off between expressiveness and reliability that should be tuned based on downstream use cases.

7 Conclusion

This project establishes a novel pathway for automating 3D engineering design by introducing **CAD-MLLM**, a unified architecture capable of interpreting multimodal signals into executable parametric history. A primary contribution of this work is the development of the “Intelligent Truncation” algorithm, which successfully bridges the data gap between static design repositories and the sequential logic required for training autoregressive agents. By effectively serializing topological constraints into language tokens and expanding the training corpus to over 197,000 valid design variations, we have demonstrated that Large Language Models can be repurposed from general text processors into specialized geometric reasoning engines capable of maintaining validity across partial and complete design states. Ultimately, this framework moves beyond simple shape generation to provide a foundation for AI assistants that can intuitively edit and construct precise, engineering-ready assets, significantly lowering the barrier to entry for parametric CAD modeling.

References

- [1] Anna C Doris, Md Ferdous Alam, Amin Heyrani Nobari, and Faez Ahmed. Cad-coder: An open-source vision-language model for computer-aided design code generation. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89220, page V03AT03A031. American Society of Mechanical Engineers, 2025.

- [2] Mohammad Sadil Khan, Sankalp Sinha, Sheikh Talha Uddin, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2cad: Generating sequential CAD designs from beginner-to-expert level text prompts. In *Advances in Neural Information Processing Systems*, volume 37, pages 7552–7579. Curran Associates, Inc., 2024.
- [3] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6772–6782, October 2021.
- [4] Haoyang Xie and Feng Ju. Text-to-cadquery: A new paradigm for cad generation with scalable large model capabilities. *arXiv preprint arXiv:2505.06507*, 2025.
- [5] Jingwei Xu, Chenyu Wang, Zibo Zhao, Wen Liu, Yi Ma, and Shenghua Gao. Cad-mllm: Unifying multimodality-conditioned cad generation with mllm, 2024.